CE510: Modern Statistics for Engineers

Winter 2026 Final Project

Completed by Brandon Auyeung for Dr. Peyman Abbaszadeh.

Let us define variables.

$S \equiv$ suspended sediment concentration, SSC $\doteq [\frac{M}{L^3}]$

$Q \equiv$ discharge $\doteq [L^3]$

$\alpha,\ \beta \equiv$ model parameters

$\sigma \equiv$ unknown measurement error

Note:

S in Excel file is given in mg/L. Q in Excel is given in cfs.

1 $\mathrm{mg/L}$ **= 0.001** $\mathrm{kg/m^3}$**. 1** cfs **= 0.0283168** $\mathrm{m^3/s}$

In [193…

```
import numpy as np
import pandas as pd
from icecream import ic
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats
import scipy.optimize as opt
import emcee
```

In [194…

```
data = pd.read_excel("Data_Final_Project.xlsx", skiprows=0)
data = data.drop(["Agency","Station ID"], axis=1)
data
```

Out[194…

	Date	Suspended sediment concentration	Discharge
0	1961-10-01	4.0	327
1	1961-10-02	4.0	315
2	1961-10-03	4.0	309
3	1961-10-04	4.0	303
4	1961-10-05	4.0	294
...	...	...	...
3646	1971-09-25	1.0	444
3647	1971-09-26	2.0	485
3648	1971-09-27	4.0	554
3649	1971-09-28	4.0	623
3650	1971-09-29	7.0	740

3651 rows × 3 columns

In [195…

```
x = data["Discharge"]
y = data["Suspended sediment concentration"]
data.describe()
```

Out[195…

	Date	Suspended sediment concentration	Discharge
count	3651	3587.000000	3651.000000
mean	1966-09-30 00:00:00	14.227209	4081.558203
min	1961-10-01 00:00:00	0.000000	186.000000
25%	1964-03-31 12:00:00	4.000000	655.000000
50%	1966-09-30 00:00:00	6.000000	2020.000000
75%	1969-03-30 12:00:00	16.000000	5270.000000
max	1971-09-29 00:00:00	452.000000	46500.000000
std	NaN	23.279191	5267.264166

a) Use the flat prior assumption for β and visualize the ensemble of curves.

$$S(Q_i|\alpha,\ \beta) = \alpha Q_i^{\beta}$$

Assuming Gaussian error in S_{obs}:

$$\mathbb{P}(Q_i S_i|\alpha,\beta,\sigma) = \frac{1}{\sqrt{2\pi\sigma^2}}\exp\left[\frac{-[S_i - S(Q_i|\alpha,\beta)]^2}{2\sigma^2}\right]$$

$$\mathcal{L} = \mathbb{P}(\vec{Q}_i\vec{S}_i|\alpha,\beta,\sigma) \propto (2\pi\sigma^2)^{-N/2}\exp\left[-\frac{1}{2\sigma^2}\sum_{i-1}^{N}[S_i - S(Q_i|\alpha,\beta)]^2\right]$$

In [196…

```
def log_likelihood(theta, x, y):
    alpha, beta, log_sigma = theta
```

```python
    sigma = np.exp(log_sigma)
    model = alpha * x ** beta
    return -0.5 * len(x) * np.log(2 * np.pi * sigma ** 2) - 0.5*sigma ** (-2) * np.sum((y - model) ** 2)

# if i dont add anything, flat prior is assumed (log(flat prior))

from scipy.optimize import minimize

np.random.seed(0)
nll = lambda *args: -log_likelihood(*args)
initial = np.array([0.5, 1, 20]) + 0.1 * np.random.randn(3)
soln = minimize(nll, initial, args=(x, y))
alpha_nll, beta_nll, sigma_nll = soln.x

print('Via MLE')
print(f"alpha = {alpha_nll:.3f}")
print(f"beta = {beta_nll:.3f}")
print(f"sigma = {sigma_nll:.3f}")
```

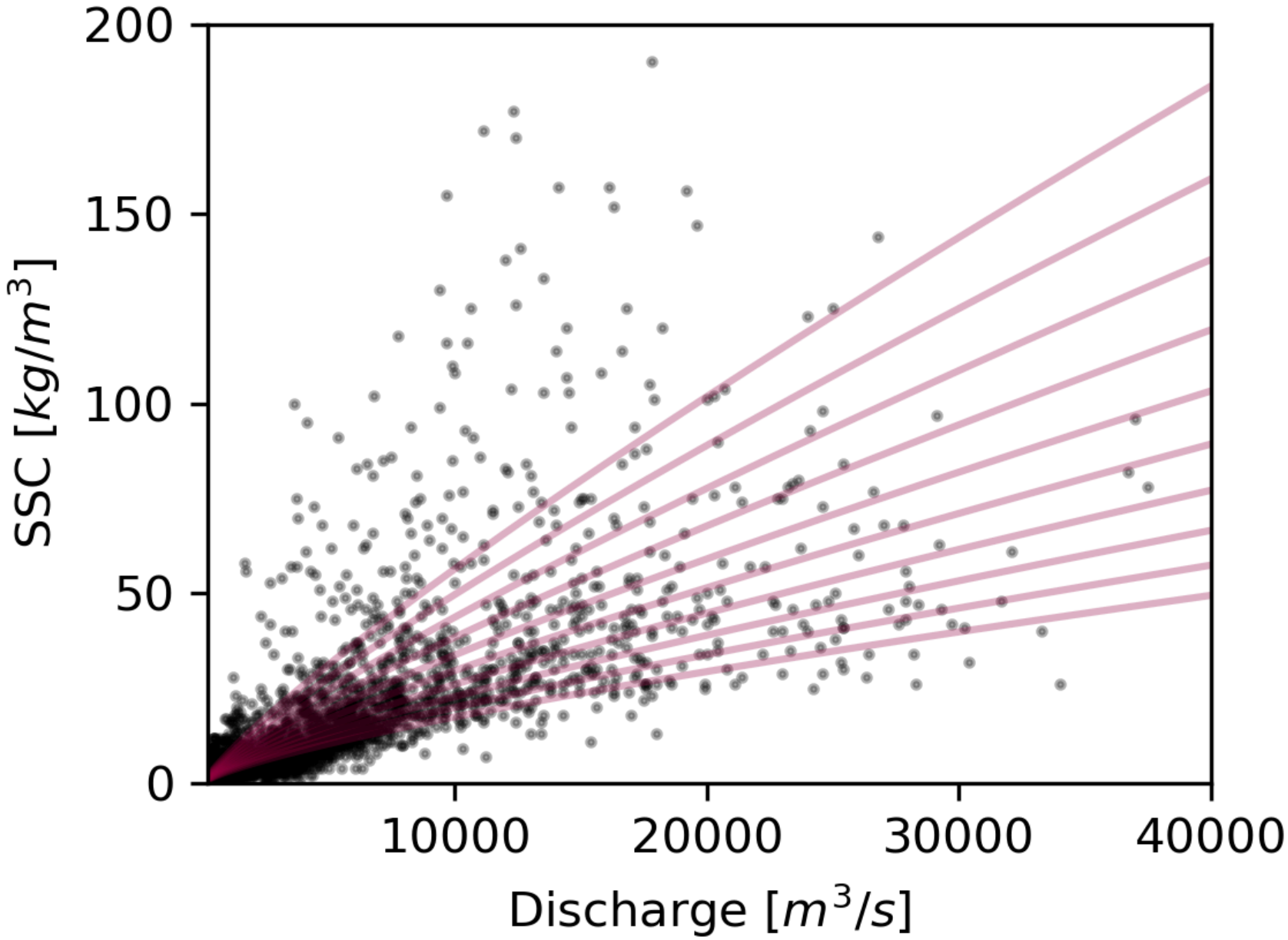
```
Via MLE
alpha = 0.022
beta = 0.793
sigma = 2.868
```

In [197…

```python
# using the alpha and beta from MLE, generate ensemble of curves and visualize
alpha_guesses = np.linspace(0.0175, 0.0225, 10)
beta_guesses = np.linspace(0.75, 0.85, 10)

fig, ax = plt.subplots(figsize=(4,3), tight_layout=True, dpi=300)

Q_line = np.linspace(200, 40000, 10000)

ax.plot(x,y, color = "black", alpha = 0.3, linestyle = "None", marker = ".", markersize = 3)

for alpha, beta in zip(alpha_guesses, beta_guesses):
    ax.plot(Q_line, alpha * Q_line ** beta, color= '#980043', alpha=0.3)

ax.set_xlabel('Discharge $[m^3/s]$')
ax.set_ylabel('SSC $[kg/m^3]$')
ax.set_xlim(200, 40000)
ax.set_ylim(0, 200)
```

Out[197…

```
(0.0, 200.0)
```



2) Use the MCMC approach to fit a curve of best-fit along with the $\pm 2\sigma$ uncertainty region.

Let's assume the following is the best choice of prior for the model parameters α and β, as well as assuming that σ follows the Jeffrey's prior.

$$\mathbb{P}(\alpha, \beta) \propto (1 + \beta^2)^{-3/2}$$

In [198…

```
def log_prior(theta):
    alpha, beta, log_sigma = theta
    sigma = np.exp(log_sigma)
```

```
    # if not (0.01 < alpha < 0.03 and -20 < beta < 20 and 0 < sigma < 40):
    #     return -np.inf
    return -1.5 * np.log(1 + beta ** 2) + (1/np.log(sigma))

def log_posterior(theta, x, y):
    return log_prior(theta) + log_likelihood(theta, x, y)
```

In [199…

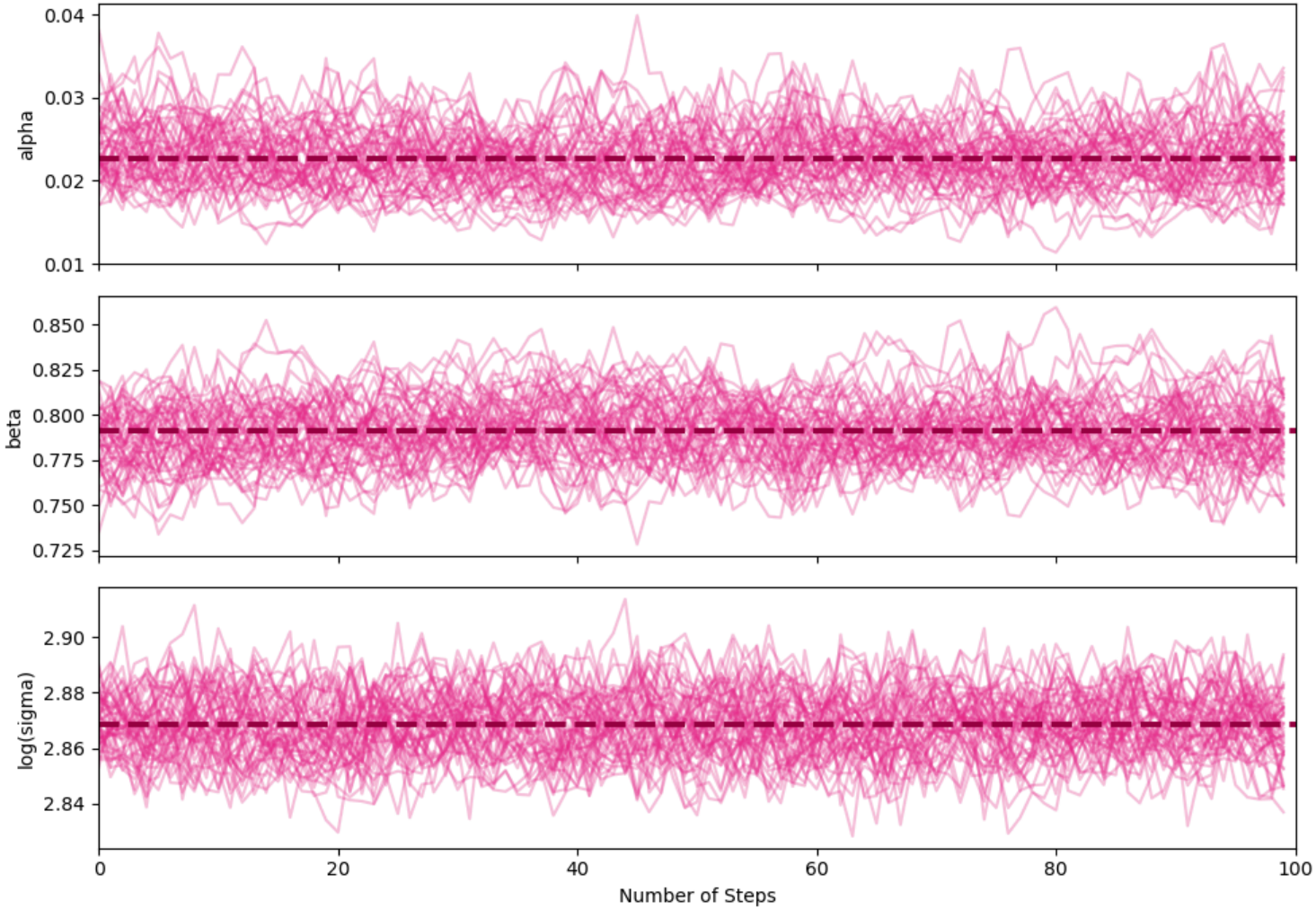
```
np.random.seed(0)

nwalkers = 50
ndim = 3
nburn = 1000
nsteps = 2000

sampler = emcee.EnsembleSampler(
    nwalkers, ndim, log_posterior, args=(x, y)
)
## sensitive to initial values
guesses = [alpha_nll, beta_nll, np.log(17)] + 0.01 * np.random.randn(nwalkers, ndim)

sampler.run_mcmc(guesses, nsteps, progress=False)
```

Out[199…

In [200…

```
fig, axes = plt.subplots(3, figsize=(10, 7), sharex=True, tight_layout=True)
samples = sampler.get_chain(discard=nburn, thin=10, flat=False)
# print(samples.shape)
labels = ["alpha", "beta", "log(sigma)"]
for i in range(ndim):
    ax = axes[i]
    for j in range(nwalkers):
        ax.plot(samples[:, j, i], "#e7298a", alpha=0.3)
    ax.set_xlim(0, len(samples[:, :, i]))
    ax.set_ylabel(labels[i])
    ax.hlines(np.mean(samples[:, :, i]), 0, len(samples[:, :, i]), color="#980043", linestyles='--', lw = 3)

axes[-1].set_xlabel("Number of Steps");

est_alpha = np.mean(samples[:, :, 0])
est_beta = np.mean(samples[:, :, 1])
est_sigma = np.mean(np.exp(samples[:, :, 2]))

print(f"alpha = {est_alpha:.3f}")
print(f"beta = {est_beta:.3f}")
print(f"sigma = {est_sigma:.3f}")
```

```
alpha = 0.023
beta = 0.791
sigma = 17.615
```



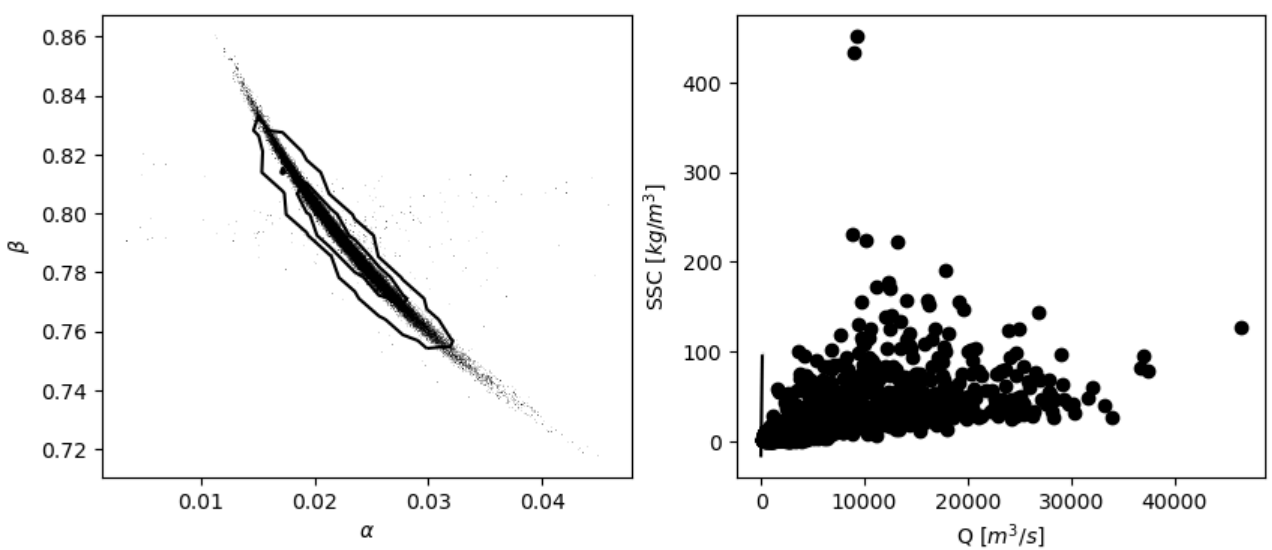
In [201…

```
# Create some convenience routines for plotting

def compute_sigma_level(trace1, trace2, nbins=20):
    """From a set of traces, bin by number of standard deviations"""
    L, xbins, ybins = np.histogram2d(trace1, trace2, nbins)

    L[L == 0] = 1E-16
    logL = np.log(L)

    shape = L.shape
    L = L.ravel()

    # obtain the indices to sort and unsort the flattened array
    i_sort = np.argsort(L)[::-1]
    i_unsort = np.argsort(i_sort)

    L_cumsum = L[i_sort].cumsum()
    L_cumsum /= L_cumsum[-1]

    xbins = 0.5 * (xbins[1:] + xbins[:-1])
    ybins = 0.5 * (ybins[1:] + ybins[:-1])
```

```python
    return xbins, ybins, L_cumsum[i_unsort].reshape(shape)

def plot_MCMC_trace(ax, xdata, ydata, trace, scatter=False, **kwargs):
    """Plot traces and contours"""
    xbins, ybins, sigma = compute_sigma_level(trace[0], trace[1])
    ax.contour(xbins, ybins, sigma.T, levels=[0.683, 0.955], **kwargs)
    if scatter:
        ax.plot(trace[0], trace[1], ',k', alpha=0.1)
    # ax.set_xlim(-1, 1)
    # ax.set_ylim(0, 1)
    ax.set_xlabel(r'$\alpha$')
    ax.set_ylabel(r'$\beta$')


def plot_MCMC_model(ax, xdata, ydata, trace):
    """Plot the linear model and 2sigma contours"""
    ax.plot(xdata, ydata, 'ok')

    alpha, beta = trace[:2]
    xfit = np.linspace(-20, 120, 10)
    yfit = alpha[:, None] + beta[:, None] * xfit
    mu = yfit.mean(0)
    sig = 2 * yfit.std(0)

    ax.plot(xfit, mu, '-k')
    ax.fill_between(xfit, mu - sig, mu + sig, color='lightgray')

    ax.set_xlabel('Q $[m^3/s]$')
    ax.set_ylabel('SSC $[kg/m^3]$')

def plot_MCMC_results(xdata, ydata, trace, colors='k'):
    """Plot both the trace and the model together"""
    fig, ax = plt.subplots(1, 2, figsize=(10, 4))
    plot_MCMC_trace(ax[0], xdata, ydata, trace, True, colors=colors)
    plot_MCMC_model(ax[1], xdata, ydata, trace)
```

In [202…

```python
emcee_trace = sampler.chain[:, :, :].reshape(-1, ndim).T
plot_MCMC_results(data['Discharge'], data['Suspended sediment concentration'], emcee_trace)
# Show the figure
# plt.tight_layout()
```

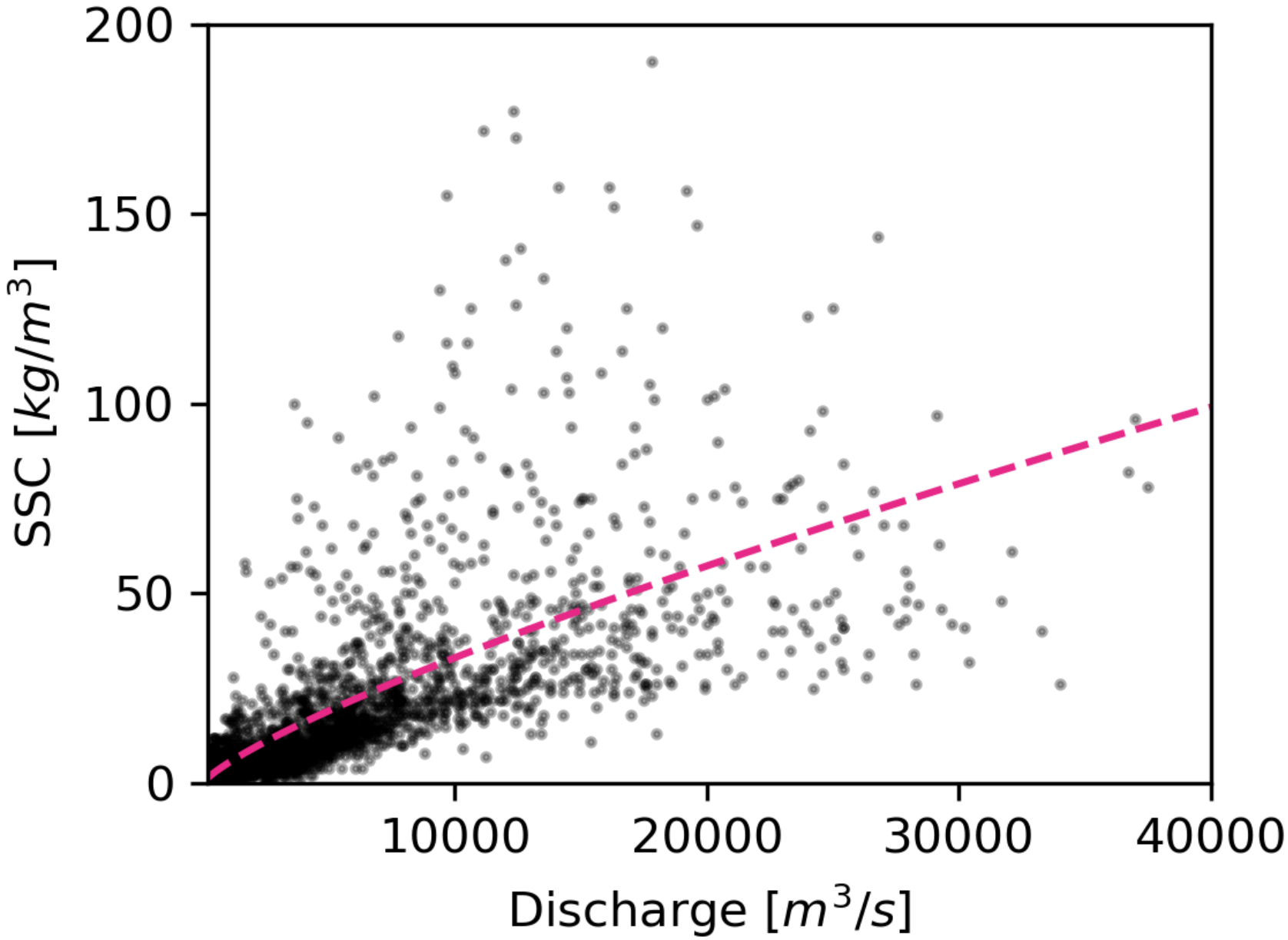


In [203…

```
fig, ax = plt.subplots(figsize=(4,3), tight_layout=True, dpi=300)
Q_line = np.linspace(200, 50000, 10000) # test muliple flow rate values

ax.plot(x,y, color = "black", alpha = 0.3, linestyle = "None", marker = ".", markersize = 3)
ax.plot(Q_line, est_alpha*Q_line**est_beta, linestyle='--', alpha = 1, color = "#e7298a")

ax.set_xlabel('Discharge $[m^3/s]$')
ax.set_ylabel('SSC $[kg/m^3]$')
ax.set_xlim(200, 40000)
ax.set_ylim(0, 200)
```

Out[203…

```
(0.0, 200.0)
```

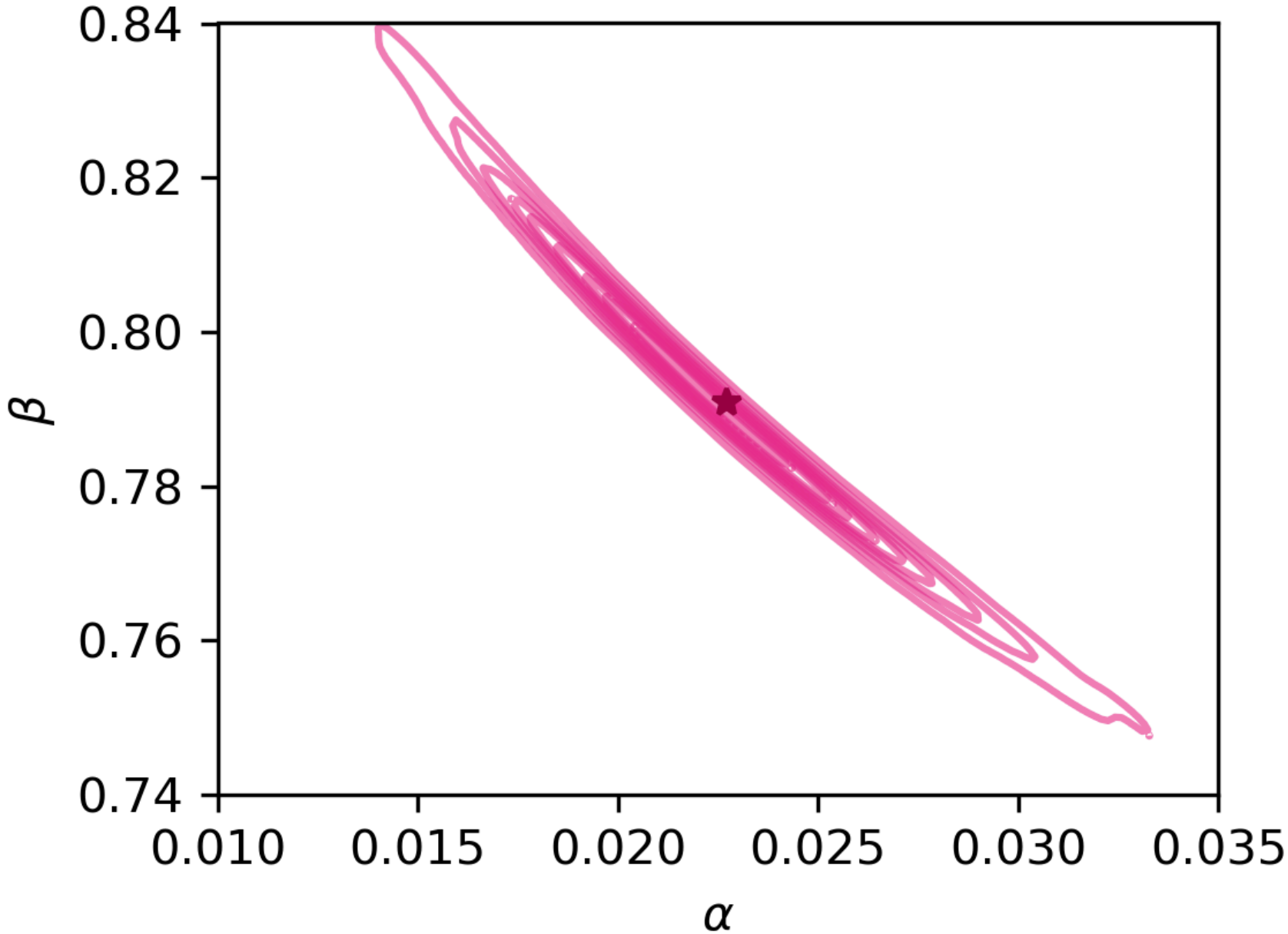


3. Use KDE to visualize the resulting traces marginalized over the uncertain parameter σ.

In [204…

```
fig, ax = plt.subplots(figsize=(4,3), tight_layout=True, dpi=300)
sns.kdeplot(x= np.ravel(samples[:, :, 0]), y = np.ravel(samples[:, :, 1]), color = "#e7298a", alpha = 0.6)
plt.xlim(0.01, 0.035)
plt.xlabel('$\\alpha$')
plt.ylim(0.74, 0.84)
plt.ylabel('$\\beta$')
plt.plot(est_alpha, est_beta, '*', color = "#980043")
```

Out[204… [<matplotlib.lines.Line2D at 0x7c3556c486e0>]



4. Use Least Squares method to find the model parameters α and β. Visualize the result similar to Figure 2.4 in lecture 2.

In [205…

```
def cost_function(alpha, beta):
 predictions = alpha * x ** beta
 residuals = y - predictions
 return np.sum(residuals**2)
```

```
initial_guess = [alpha_nll, beta_nll] # Initial guess for a, b, and c
result = minimize(lambda params: cost_function(params[0], params[1]), initial_guess)
alpha_ls, beta_ls = result.x
```

In [206…

```
print('Via Least Squares')
print(f"    alpha = {alpha_ls:.3f}")
print(f"    beta = {beta_ls:.3f}")

print('Via Maximum Likelihood Estimation')
print(f"    alpha = {alpha_nll:.3f}")
print(f"    beta = {beta_nll:.3f}")

print('Via MCMC')
print(f"    alpha = {np.mean(samples[:, :, 0]):.3f}")
print(f"    beta = {np.mean(samples[:, :, 1]):.3f}")
```

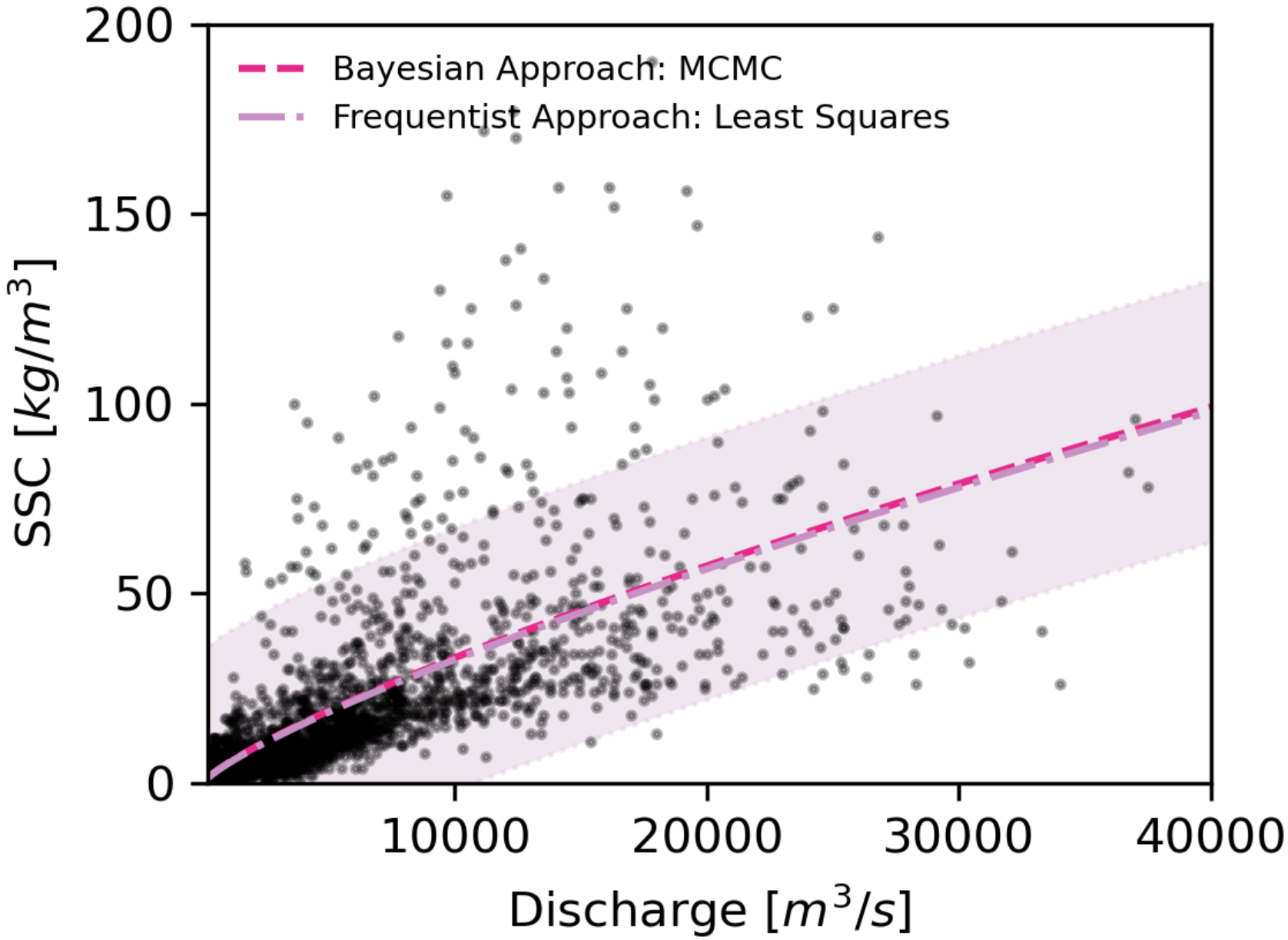
```
Via Least Squares
    alpha = 0.022
    beta = 0.793
Via Maximum Likelihood Estimation
    alpha = 0.022
    beta = 0.793
Via MCMC
    alpha = 0.023
    beta = 0.791
```

In [207…

```
fig, ax = plt.subplots(figsize=(4,3), tight_layout=True, dpi=300)
Q_line = np.linspace(200, 50000, 10000) # test muliple flow rate values

ax.plot(x,y, color = "black", alpha = 0.3, linestyle = "None", marker = ".", markersize = 3)
ax.plot(Q_line, est_alpha*Q_line**est_beta, '--', alpha = 1, color = "#e7298a", label = "Bayesian Approach: MCMC")

ax.plot(Q_line, alpha_ls*Q_line**beta_ls, linestyle='-.', alpha = 1, color ='#c994c7', label = "Frequentist Approach: Least Squares")
ax.fill_between(Q_line, alpha_ls*Q_line**beta_ls - 1.96 * est_sigma, alpha_ls*Q_line**beta_ls + 1.96 * est_sigma,\
                linestyle=':', color='#c994c7', alpha=0.2)

plt.legend(frameon = False, loc = 'upper left', fontsize = 7)

ax.set_xlabel('Discharge $[m^3/s]$')
ax.set_ylabel('SSC $[kg/m^3]$')
ax.set_xlim(200, 40000)
ax.set_ylim(0, 200)
```

Out[207…

```
(0.0, 200.0)
```



5. Compare the results from Bayesian approach (part b) with frequentist approach (part d).

Results from the Bayesian approach and Frequentist approach are very similar. However, only the Bayesian approach is able to estimate a value of σ.